An Agent's Life as a Board Game:
A Friendly Markov Chain Primer for the Code Coverage Experiment

Mark Pollack

March 2026

Contents

1 The Big Idea 2

2 The 9 States 2

3 Following the Flow: Sankey Diagrams 3

4 The Probability Matrix: Where Does the Agent Go Next? 4

5 Absorbing States: The Game Has to End 6

6 The Fundamental Matrix: Expected Visits 6

7 Loop Amplification: Measuring Wasted Effort 7

8 The JAR Cluster: A Named, Quantified Problem 8

9 Intervention Delta: Knowledge as a Probability Editor 9

10 Context Window as Attention Budget 9

11 Thrashing Loop Detection 10

12 The Punchline: TPE Works 11

13 What's Next 12

1 The Big Idea

Imagine you're watching an AI agent work. It reads a file. Then it runs a build. Then it reads another file. Then it runs the build again. And again.

What you're watching is a *sequence of tool calls* — and that sequence has structure. It's not random, and it's not deterministic. It's somewhere in between. That "somewhere in between" has a name: a **Markov chain**.

The Board Game Analogy

Think of the agent as a player on a board game. The board has squares (states), and at each turn the player rolls dice and moves to a new square. The key rule: *where you go next depends only on where you are now*, not on the full history of how you got there.

That rule — "only the current square matters" — is the Markov property. For an LLM agent, it's approximately true: the model is stateless, and the full context (current square) is re-injected at every step. The agent "forgets" its history except what's in the prompt.

Why It Matters

If the agent's moves follow a Markov chain, we can:

- **Measure** how much time it spends in each state
- **Identify** which loops are wasting effort
- **Predict** how many steps it will take before it finishes
- **Intervene** by changing the probabilities (Transition Probability Engineering)

In this experiment we recorded 1,413 tool calls across 9 variants and 5 benchmark items. We then modeled the sequence as a Markov chain with 9 states. Here's what we found.

2 The 9 States

We classified every tool call into one of 9 semantic states based on the tool name and target. Table 1 describes each state in plain English.

State	Symbol	What it means
EXPLORE	E	Listing directories, navigating the project structure
READ_SOURCE	RS	Reading production Java source files
READ_KB	RK	Reading knowledge base files (curated docs)
READ_SAE	RA	Reading pre-computed static analysis (SAE variant)
WRITE_TEST	WT	Writing or editing test files
BUILD	B	Running `mvn test` or `mvnw test`
JAR_INSPECT	JI	Decompiling or reading class files in `.m2` cache
JAR_FIND	JF	Searching the `.m2` repository for JARs
FIX_TEST	FT	Editing tests after a build failure

Table 1: The 9 semantic states and their plain-English meanings.

Figure 1 shows how frequently each state appears across all 9 variants.

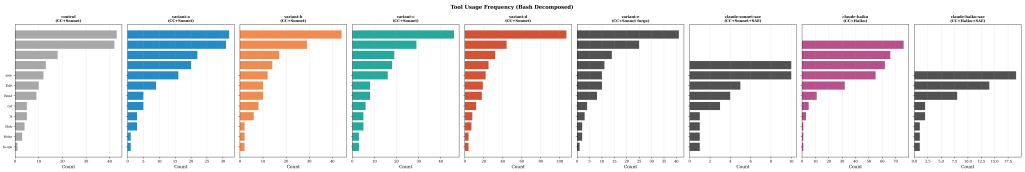


Figure 1: Raw tool call counts by state and variant. The JAR_INSPECT spike in variant-a is the clearest signal in the raw data.

Table 2 shows the fraction of tool calls in each state, per variant. The JAR cluster (JAR_INSPECT + JAR_FIND) ranges from 0% (haiku-sae) to 66% (sonnet-sae) — a 66× range driven entirely by experimental conditions.

Variant	EXPLORE	READ_SRC	READ_KB	READ_SAE	WRITE_TEST	BUILD	JAR_INSP	JAR_FIND	FIX_TEST
control	28.2%	28.9%	0.0%	0.0%	12.1%	8.7%	6.7%	9.4%	6.0%
variant-a	13.5%	14.9%	0.0%	0.0%	10.8%	22.3%	23.6%	11.5%	3.4%
variant-b	14.7%	18.0%	11.3%	0.0%	11.3%	19.3%	12.0%	6.7%	6.7%
variant-c	13.0%	16.0%	12.3%	0.0%	9.9%	17.9%	15.4%	10.5%	4.9%
variant-d	11.9%	27.0%	10.5%	0.0%	11.2%	15.4%	11.2%	6.3%	6.3%
variant-e	10.6%	29.3%	4.1%	0.0%	11.4%	21.1%	9.8%	5.7%	8.1%
sonnet-sae	11.4%	8.6%	0.0%	2.9%	2.9%	8.6%	42.9%	22.9%	0.0%
claude-haiku	15.0%	19.5%	1.6%	0.0%	19.8%	24.3%	0.6%	1.6%	17.6%
haiku-sae	12.5%	14.6%	0.0%	2.1%	2.1%	39.6%	0.0%	0.0%	29.2%

Table 2: State distribution by variant (fraction of total tool calls).

3 Following the Flow: Sankey Diagrams

Before we get into the math, let's look at the flow directly. A Sankey diagram shows transitions: each arc represents a transition from one state to the next, and the *width of the arc* represents how often that transition happens.

Figure 2 shows all four main variants side by side — the most intuitive way to see the differences.

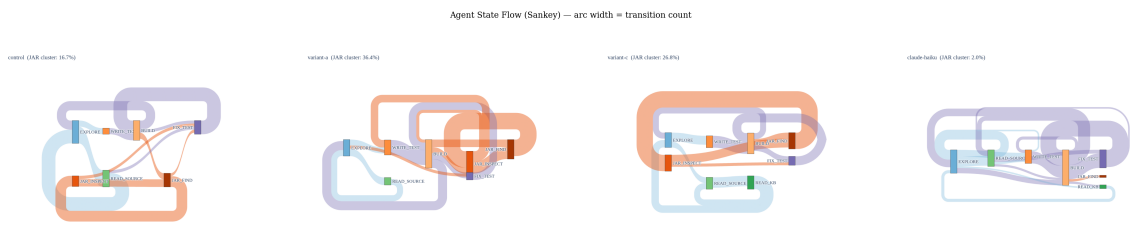
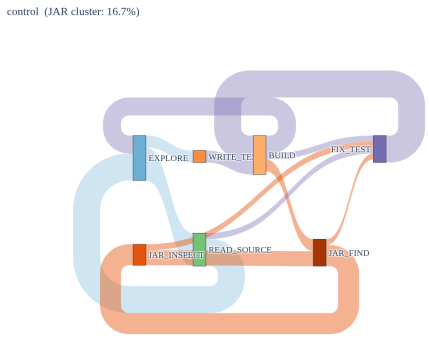
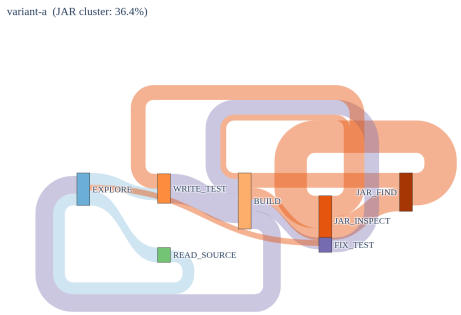


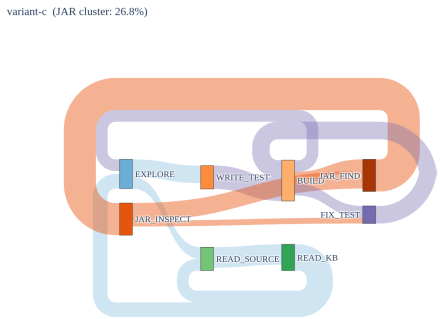
Figure 2: Sankey flow comparison across control, variant-a, variant-b, and variant-c. Wide arcs = common transitions. The JAR cluster dominates variant-a.



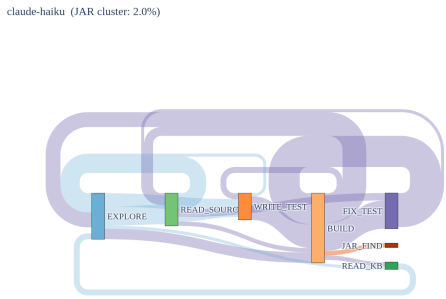
(a) Control: balanced, exploratory flow



(b) Variant-a: JAR cluster dominates



(c) Variant-c: KB loop appears (READ_KB)



(d) Haiku: BUILD/FIX loop dominates

Figure 3: Individual Sankey diagrams. Each variant has a different "personality." Variant-c adds the READ_KB loop. Haiku is stuck in compile–fix cycles.

Reading these diagrams: follow the thick arcs. In variant-a, the biggest arcs flow *into and out of JAR_INSPECT* — the agent is stuck searching the Maven cache for class definitions. In haiku, the thick arcs go BUILD → FIX_TEST → BUILD → FIX_TEST. . . — the agent is stuck in a compile–fix cycle it can't escape.

4 The Probability Matrix: Where Does the Agent Go Next?

The Sankey gives us intuition. Now let's put numbers on it.

A **transition probability matrix** P is a grid where:

- Each row is a "from" state (where the agent is now)
- Each column is a "to" state (where it goes next)
- $P(i, j)$ = probability of going from state i to state j
- Each row sums to 1 (the agent has to go *somewhere*)

The Markov Property (the key assumption)

We're computing these probabilities by counting: of all the times the agent was in state i, what fraction of the time did it go to state j next?

This is valid if the Markov property holds: *the next state depends only on the current state, not on the full history.* For an LLM agent this is approximately true — the model itself is stateless, and the context window re-injects the current situation at every step. History matters only insofar as it's encoded in the current context.

The Self-Loop: A Red Flag

The diagonal of P is the self-loop: $P(i, i)$ = probability of staying in state i. A high diagonal entry means the agent tends to repeat the same action. Some repetition is fine (reading multiple source files). Too much repetition is **thrashing**.

Key number: In variant-a, the JAR_INSPECT self-loop is $P(\text{JI}, \text{JI}) = 0.514$. That's a coin flip — half the time after inspecting a JAR, the agent inspects another JAR. Combined with the JAR_INSPECT → JAR_FIND transition, the agent stays in the JAR cluster with probability **0.743** at each step. It barely escapes.

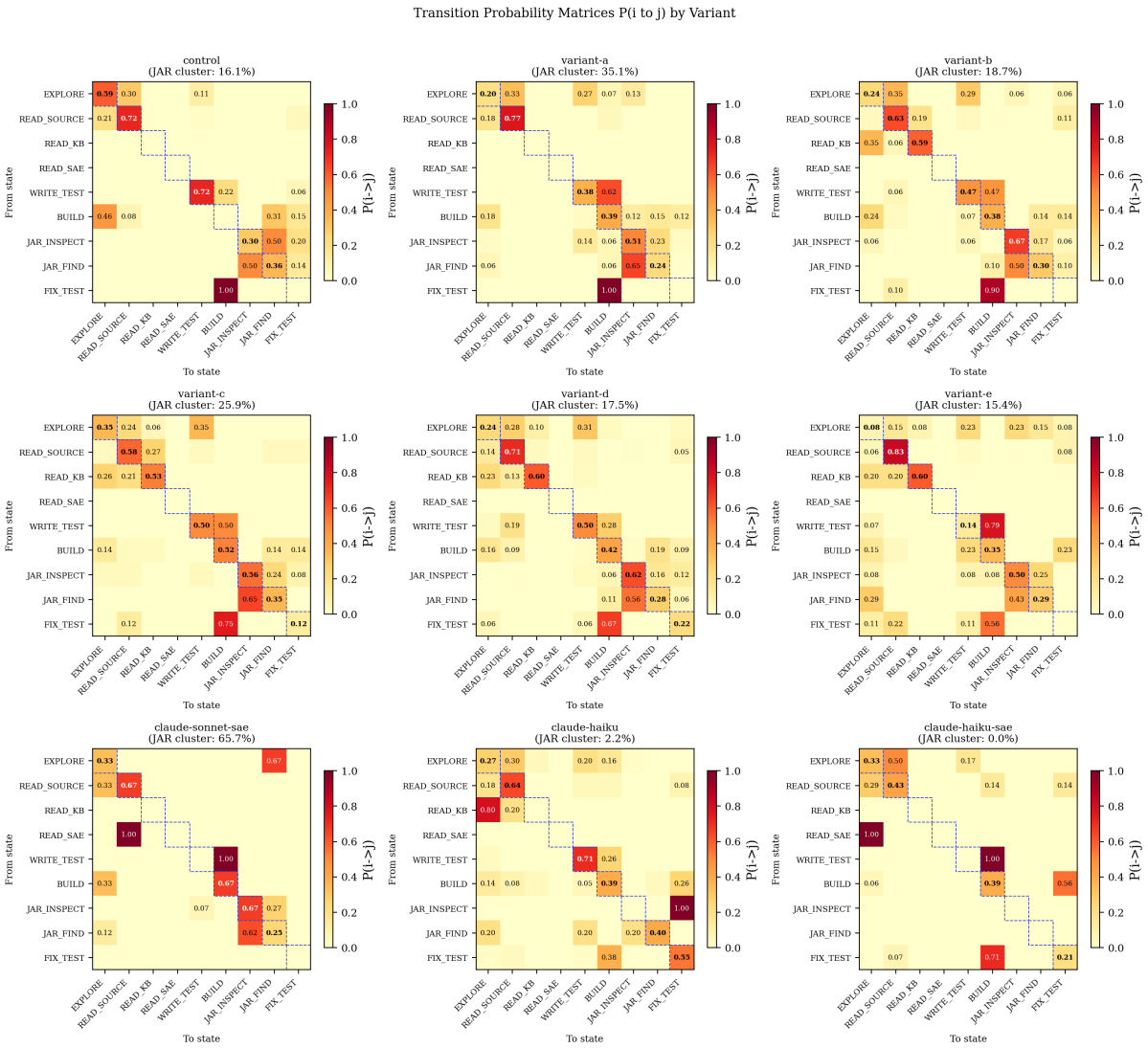


Figure 4: Transition probability matrices for all variants. The diagonal shows self-loops (red = high probability of staying). Variant-a's JAR_INSPECT row is hot. Haiku's BUILD and FIX_TEST rows are hot.

5 Absorbing States: The Game Has to End

Not every Markov chain runs forever. Our agent eventually terminates — either with a passing build (success) or with a timeout/error (failure).

Absorbing states are states the agent enters and never leaves: BUILD_SUCCESS (win) and TIMEOUT/ERROR (lose). Everything else is a **transient state** — a square the agent might visit, but will eventually leave.

> *Think of absorbing states as the exits on the board. Step on one, game over. Some exits are marked "win," others "lose." The agent is trying to find the win exit without spending too many turns wandering.*

This matters for the math. A popular tool for analyzing Markov chains is **PageRank** — used by Google to rank web pages. But PageRank assumes the chain runs forever (ergodic chains). Our agent's runs are *finite*: they end. PageRank gives wrong answers for absorbing chains. The right tool is the **fundamental matrix**.

6 The Fundamental Matrix: Expected Visits

Here's the central equation of absorbing Markov chain theory:

$$N = (I - Q)^{-1}$$

Where:

- Q = the sub-matrix of P containing only transitions between *transient* states (remove the absorbing state rows and columns)
- I = the identity matrix (ones on diagonal, zeros elsewhere)
- N = the **fundamental matrix**

What N tells you: Entry $N(i, j)$ is the expected number of times the agent visits state j, given that it starts in state i, before it reaches an absorbing state.

The diagonal of N: $N(i, i)$ = expected total visits to state i before exit. A large diagonal entry means the agent spends a lot of time in that state. For thrashing states (like JAR_INSPECT in variant-a), this number is high.

Row sums of N: The total expected steps before absorption. This is the expected *length* of a run.

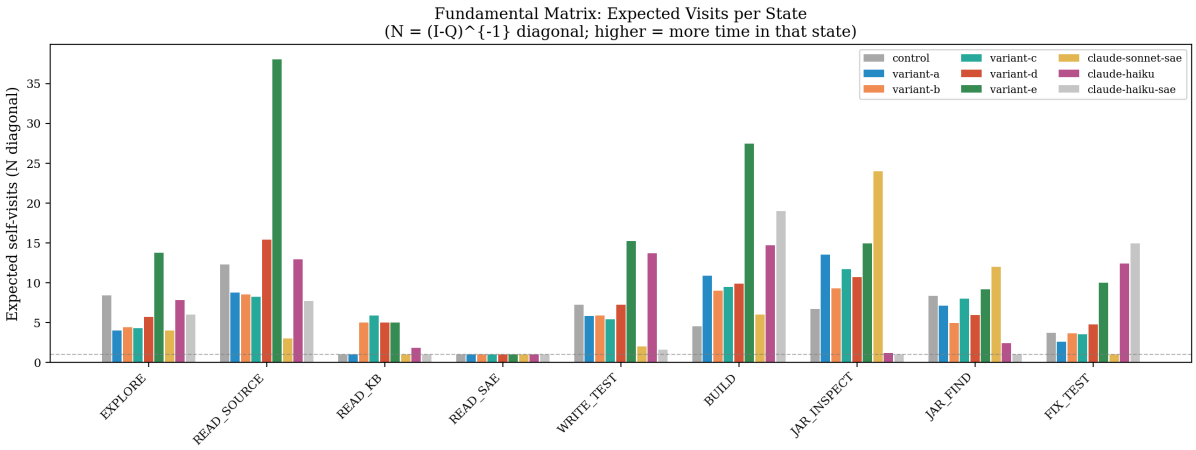


Figure 5: Fundamental matrix N: expected visits per state, per variant. Large values on the diagonal indicate loops. The JAR_INSPECT diagonal is highest for variant-a.

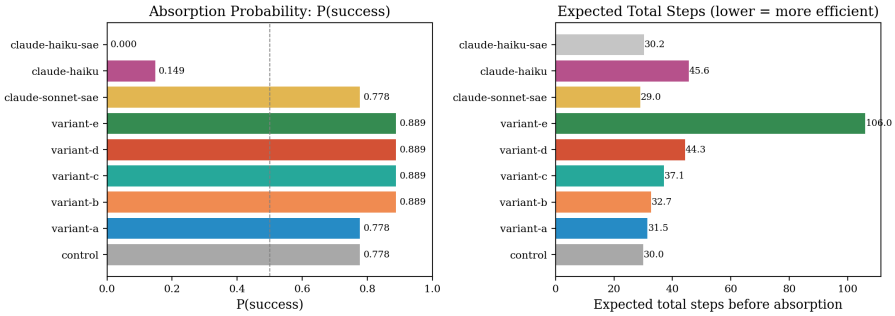


Figure 6: Summary: expected total steps and P(success) per variant. The KB variants (b, c, d) all achieve P(success)=0.889 despite different step counts.

Key Results

Variant	Expected Steps	P(success)	Notes
control	30.0	0.778	Baseline (naive prompt)
variant-a	31.5	0.778	Hardened prompt, no KB
variant-b	32.7	0.889	3-file KB
variant-c	37.1	0.889	Full KB
variant-d	44.3	0.889	Extended run
claude-haiku	45.6	0.149	Cheap model, poor outcomes
haiku-sae	30.2	0.000	SAE preprocessing, zero successes

Table 3: Fundamental matrix summary: expected total steps and absorption probability.

Surprising finding: variant-c has *more* expected steps than variant-a (37.1 vs 31.5). Does that mean the KB made things worse?

No. The fundamental matrix counts *all* visits, including productive ones. Variant-c adds READ_KB steps — genuinely useful reading. More steps + same P(success) = more productive work, not more waste. The right metric for thrashing is the **JAR cluster percentage**: 35.1% (variant-a) → 25.9% (variant-c). That's the improvement.

7 Loop Amplification: Measuring Wasted Effort

Expected steps from N tells us total time. But we want to know: *how much of that time is wasted*?

Loop amplification answers this:

$$\text{amplification}(i) = \frac{N(i,i)}{\bar{v}_i^{\text{success}}}$$

where $\bar{v}_i^{\text{success}}$ is the average number of visits to state i on runs that *succeed*.

Amplification > 1 means the agent visits state i more than the minimum needed to succeed. Amplification $= 5$ means the agent takes $5\times$ as many steps in that state as a successful run requires.

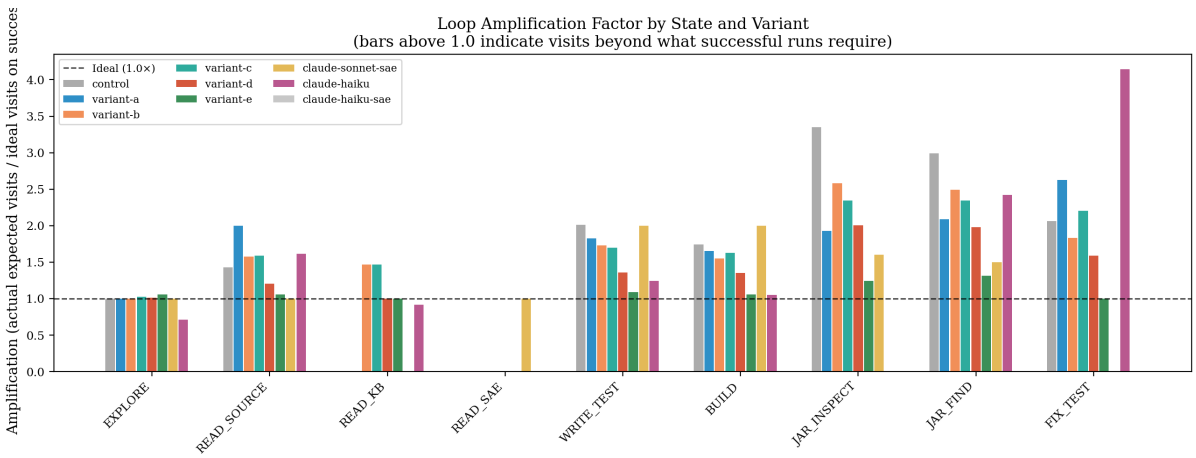


Figure 7: Loop amplification per state and variant. Variant-a peaks at JAR_INSPECT (stuck in JAR cluster). Haiku-sae peaks at BUILD/FIX_TEST (stuck in compile–fix loop).

The peak shifts: different variants have different "worst states."

- **Variant-a**: peak at JAR_INSPECT — agent searches Maven cache excessively
- **Claude-haiku**: peak at BUILD/FIX_TEST — agent can't fix compile errors
- **Variant-b/c**: peaks flatten — KB redirects the agent to productive paths

The peak tells you *exactly which loop to fix*. That's actionable.

8 The JAR Cluster: A Named, Quantified Problem

JAR_INSPECT and JAR_FIND are two states that form a tight loop. When the agent is in this cluster, it's decompiling class files and searching the `~/.m2` Maven cache, trying to understand a class it can't find in the source.

Why does this happen? The agent knows something is wrong (a compile error mentions an unknown class) but it can't find the source code. So it searches the binary artifacts instead.

The paradox: the hardened prompt (variant-a) increases JAR cluster from 16.1% to 35.1%.

Why? The hardened prompt teaches the agent to be thorough. It knows it *should* find the class definition. But without knowing *where* to look (which the KB provides), it searches everywhere — especially the last resort: JARs. "Awareness without resolution = maximum searching."

The fix: variant-b (3-file KB) drops JAR cluster to 18.7%. The KB tells the agent exactly which files contain class definitions. The agent reads the KB and goes directly to the source.

Variant	JAR Cluster %	Change
control	16.1%	—
variant-a	35.1%	+19.0% (awareness without resolution)
variant-b	18.7%	−16.4% (3-file KB fixes it)
variant-c	25.9%	+7.2% (full KB adds useful reads)

Table 4: JAR cluster percentage across variants. The paradox: hardened prompt makes JAR thrashing worse. Knowledge fixes it.

9 Intervention Delta: Knowledge as a Probability Editor

We've been comparing states in isolation. Now let's ask: *what does each intervention actually change?*

An **intervention delta** is the difference between two transition matrices:

$$\Delta P = P_{\text{after}} - P_{\text{before}}$$

Green cells = transitions that became more likely. Red cells = transitions that became less likely. This shows exactly which edges in the Markov chain were edited by each intervention.

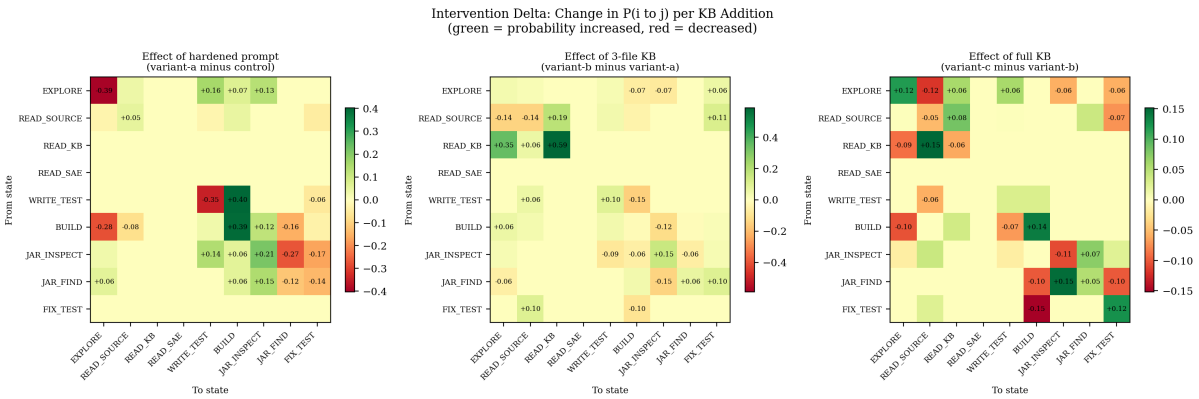


Figure 8: Intervention deltas. Three panels: (1) Hardened prompt effect (variant-a − control), (2) 3-file KB effect (variant-b − variant-a), (3) Full KB effect (variant-c − variant-b). Green = probability increased. Red = probability decreased.

The key signal: When the 3-file KB is added (panel 2), the READ_KB → READ_KB transition increases by +0.588.

What does that mean? The agent starts reading the knowledge base — and keeps reading it. It's not just a one-shot lookup. The KB pulls the agent into a sustained reading loop, replacing the JAR_INSPECT loop with a READ_KB loop. One loop for another — but this one is productive.

Transition Probability Engineering

This is the practitioner's insight: **every prompt change, every knowledge file, every structural constraint** is editing the entries of the P matrix.

We call this **Transition Probability Engineering (TPE)**. You don't need to think in matrices. You just need to ask: "What transitions am I trying to encourage, and which am I trying to suppress?"

The KB encourages READ_KB transitions. The hardened prompt encourages JAR searching. SAE preprocessing is supposed to encourage direct reading — and for Sonnet it does, but at the cost of hitting the JAR cluster heavily.

10 Context Window as Attention Budget

Theory: When an agent loops, it generates large amounts of context (tool results). Eventually the prompt gets so long that earlier instructions are "crowded out" by recent tool results (Liu et al. 2023, "lost in the middle"). The agent starts to forget what it was trying to do.

If this theory is right, we'd expect: agents that loop more also burn more tokens.

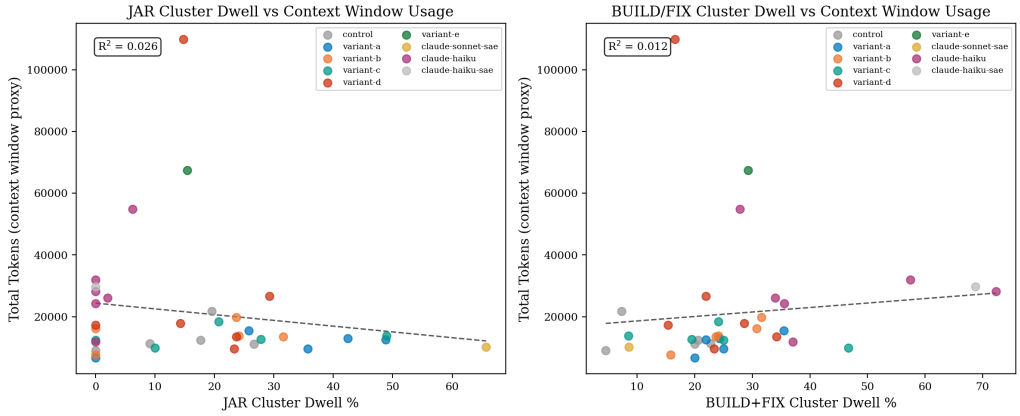


Figure 9: Scatter: JAR cluster dwell vs total tokens per run (all variants pooled). Weak correlation: $R^2 = 0.026$. The haiku cluster (upper right) burns 29k tokens vs Sonnet's 13k.

Our result: $R^2 = 0.026$ — very weak. We only had 35 data points (aggregate tokens, not per-turn tracking), which isn't enough to confirm or deny the theory.

But: claude-haiku burns 29k tokens on average vs Sonnet's 13k. Haiku thrashes more, and it burns more tokens. That's consistent with the theory — just not statistically confirmed.

The hypothesis stands. We need per-turn token tracking to test it properly.

11 Thrashing Loop Detection

Figure 10 shows the Strongly Connected Component (SCC) analysis: which groups of states form tight cycles, and what fraction of time is spent there?

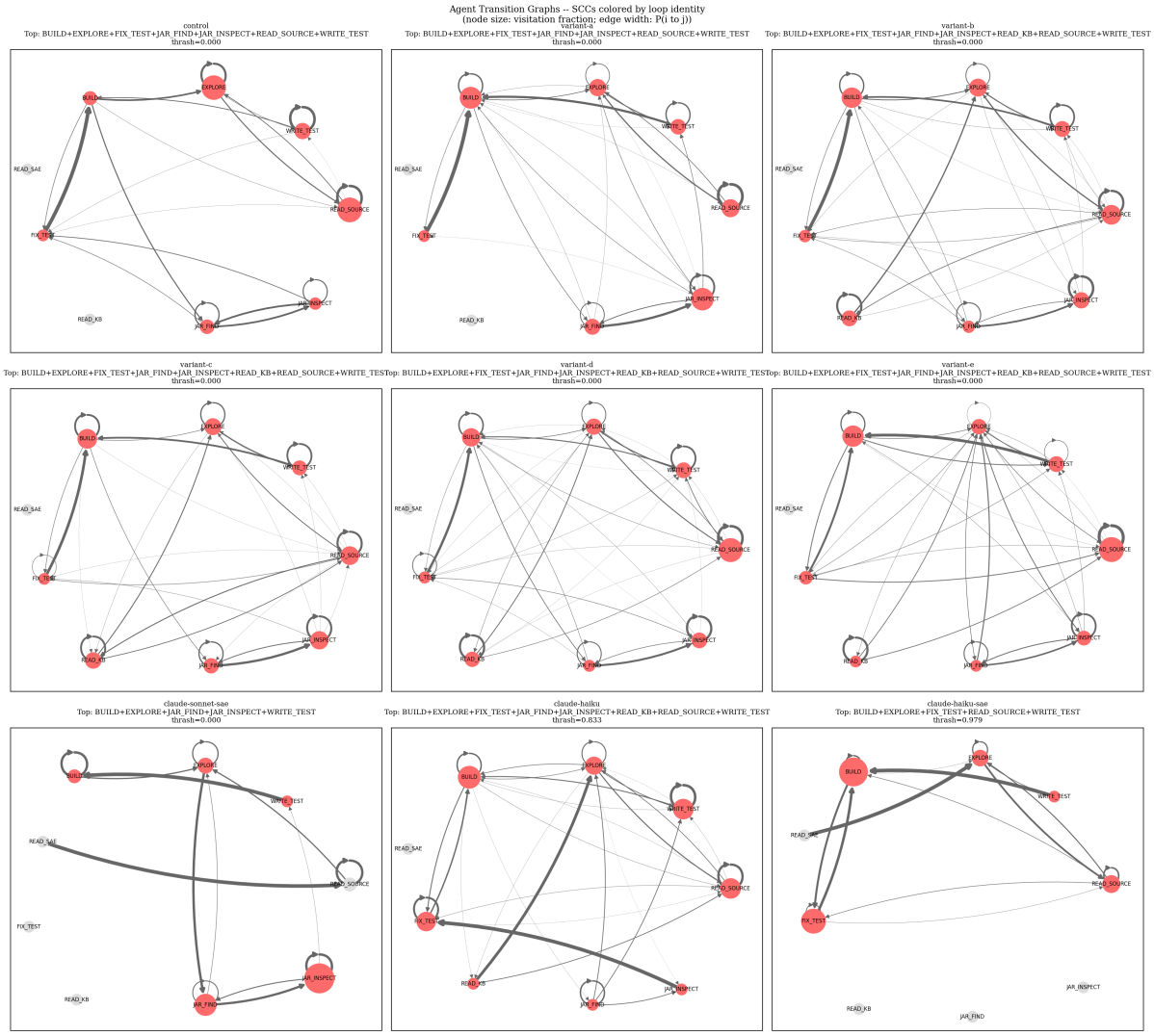


Figure 10: SCC loop detection. Control and Sonnet variants: one large SCC, low failure rate (thrash score ≈ 0). Claude-haiku: 83% failure rate within its SCC (thrash score = 0.833). Haiku-sae: 100% failure (thrash score = 0.979).

The **thrash score** = visitation mass $\times$ P(failure | SCC). A score near 1 means the agent loops heavily in states that predict failure.

Claude-haiku's thrash score of 0.833 is the smoking gun: the agent is looping, and those loops predict failure. Haiku-sae at 0.979 is even worse — near-perfect failure prediction.

Note: all Sonnet variants have thrash score ≈ 0 despite looping. They loop productively and recover. Haiku cannot.

12 The Punchline: TPE Works

Let's bring it all together.

Knowledge moves the needle:

- P(success) with no KB: 0.778
- P(success) with any KB: 0.889
- Same model, same prompt, just better information

Model is a floor, not a ceiling:

- Haiku P(success): 0.149, regardless of KB variant
- This is the model floor: below a capability threshold, more knowledge doesn't help
- But Sonnet + knowledge beats Sonnet alone on every metric that matters

The equation:

$$\text{knowledge} + \text{structured execution} > \text{model}$$

For most practical problems, targeted intervention (TPE) outperforms model upgrades. You can edit the transition matrix without buying a bigger model. That's the thesis, confirmed by the data.

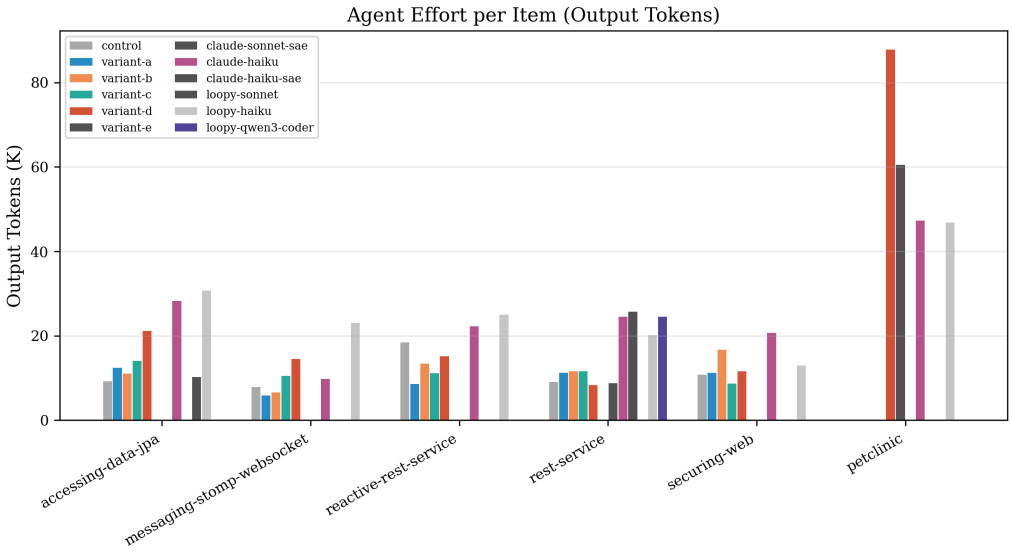


Figure 11: Distribution of run lengths by variant. Control and variant-a have similar distributions. KB variants (b/c/d) have longer but more productive runs. Haiku's distribution extends far right — it's spending time failing.

13 What's Next

The Markov analysis pipeline is now built and reusable. Here's what we'd add with more time:

1. **Per-turn token tracking**: Record tokens at each turn, not just total. Enables a proper test of the context window hypothesis (R^2 with 1000+ data points).
2. **Per-outcome transition matrices**: Separate P_{success} from P_{failure}. Which transitions predict success, which predict failure? Actionable: strengthen edges that appear on winning paths.
3. **Liquibase → Flyway experiment**: New domain, same pipeline, new data. Does the JAR cluster pattern hold in a different migration task? Replication is how we know the pattern is real.
4. **Markov analysis as a standard report**: Ship this analysis with every new agent experiment. It costs 200 lines of Python and produces more insight than the raw success rates.

Summary: We modeled 1,413 agent tool calls as an absorbing Markov chain with 9 states. The fundamental matrix revealed expected step counts and success probabilities. Loop amplification identified which loops to fix. Intervention deltas showed exactly which transitions each knowledge injection changed. And the data confirmed the thesis: knowledge-directed execution moves success rates from 0.778 to 0.889, without changing the model.